EN3160 Assignment 2: Fitting and Alignment

NAHG Nettikumara | 230436X

GitHub profile | Repository

In [1]:
```
from pathlib import Path
import json
import cv2 as cv
import numpy as np
from scipy.ndimage import gaussian_laplace, maximum_filter
from scipy.optimize import least_squares
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

ROOT = Path.cwd()
DATA = ROOT / 'a2images' / 'a2images'
OUT = ROOT / 'results'
OUT.mkdir(exist_ok=True)


from IPython.display import display, Image, Markdown
assert DATA.exists(), 'Run this notebook from the assignment folder.'
print('OpenCV:', cv.__version__, '| NumPy:', np.__version__)
```

```
OpenCV: 4.10.0 | NumPy: 1.26.4
```

In [2]:
```
def savefig(name):
    plt.tight_layout()
    path = OUT / name
    plt.savefig(path, dpi=170, bbox_inches='tight')
    plt.close()
    display(Image(filename=str(path), width=950))
```

1. Sunflower blob detection using LoG

The image is read at quarter resolution as required. A positive, scale-normalized Laplacian detects dark central disks; a yellow annulus rejects many foliage blobs. Extrema are selected jointly across position and scale, then nearby candidates are suppressed. No Hough circle detector is used.

The reported radius is the **dark central disk radius**, not the outer petal radius. The search uses 35 geometric scales from σ = 1.5 to 18 reduced-image pixels.

In [3]:
```
def blobs():
    im = cv.imread(str(DATA/'the_berry_farms_sunflower_field.jpeg'),
cv.IMREAD_REDUCED_COLOR_4)
    rgb = cv.cvtColor(im, cv.COLOR_BGR2RGB).astype(float)/255
    gray = cv.cvtColor(im, cv.COLOR_BGR2GRAY).astype(float)/255
    yellow = np.maximum(0, np.minimum(rgb[:,:,0], rgb[:,:,1])-rgb[:,:,2])
    sigmas = np.geomspace(1.5, 18, 35)
    # Positive LoG extrema detect the dark central disks, rather than petals.
    stack = np.stack([s*s*gaussian_laplace(gray, s) for s in sigmas])
    maxima = (stack == maximum_filter(stack, size=(3,3,3))) & (stack > .10)
    maxima[[0,-1]] = False
```

```
    yy,xx = np.indices(gray.shape)
    candidates=[]
    for k,y,x in np.argwhere(maxima):
        r=float(np.sqrt(2)*sigmas[k])
        if x-r<0 or y-r<0 or x+r>=gray.shape[1] or y+r>=gray.shape[0]: continue
        distance=np.hypot(xx-x,yy-y)
        ring=(distance>r)&(distance<1.9*r)
        ring_yellow=float(yellow[ring].mean())
        if ring_yellow<.4: continue

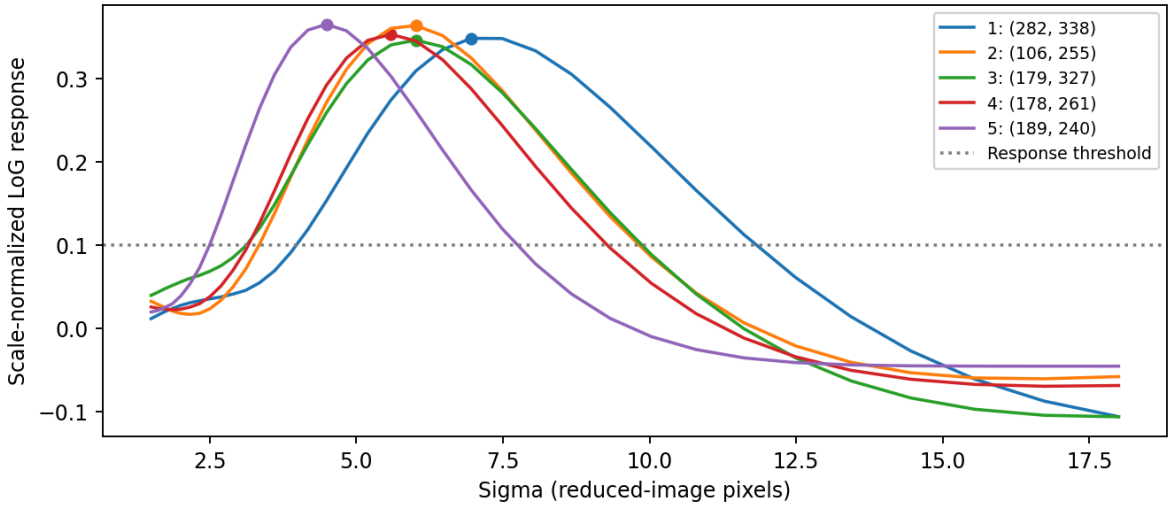
candidates.append((float(stack[k,y,x]),int(x),int(y),r,float(sigmas[k]),ring_yellow))
    kept=[]
    for c in sorted(candidates, reverse=True):
        if all(np.hypot(c[1]-d[1],c[2]-d[2])>.8*(c[3]+d[3]) for d in kept): kept.append(c)
    largest=sorted(kept,key=lambda c:c[3],reverse=True)[:5]
    fig,ax=plt.subplots(figsize=(7,7)); ax.imshow(rgb); ax.axis('off')
    for c in kept: ax.add_patch(plt.Circle(c[1:3],c[3],fill=False,color='lime',lw=1))
    for i,c in enumerate(largest): ax.text(c[1],c[2],str(i+1),color='red',fontsize=10)
    ax.set_title(f'Scale-normalized LoG: {len(kept)} dark-centre detections')
    savefig('q1_blobs.png')
    fig,ax=plt.subplots(figsize=(8,3.6))
    for i,c in enumerate(largest):
        ax.plot(sigmas,stack[:,c[2],c[1]],label=f'{i+1}: ({c[1]}, {c[2]})')
        ax.scatter([c[4]],[c[0]],s=25)
    ax.axhline(.10,c='gray',ls=':',label='Response threshold')
    ax.set(xlabel='Sigma (reduced-image pixels)',ylabel='Scale-normalized LoG response')
    ax.legend(fontsize=8); savefig('q1_scale_profiles.png')
    return {'sigma_range':[float(sigmas[0]),float(sigmas[-1])], 'scales':len(sigmas),
            'threshold':.10,'yellow_ring_threshold':.4,'ring_outer_radius_factor':1.9,
            'suppression_distance_factor':.8,'target':'dark central disks surrounded by
yellow petals',
            'detections':len(kept),'largest':
[{'x':c[1],'y':c[2],'radius':c[3],'sigma':c[4],'response':c[0],'yellow_ring_mean':c[5]} for
c in largest]}
```

In [4]:

```
q1 = blobs()
print(f"Retained candidate centres: {q1['detections']}")
print(f"Sigma search range: {q1['sigma_range']} (reduced-image pixels)")
print("Five largest detected central disks:")
print("Coordinates, sigma and radii below are in pixels of the 360 x 360 reduced image.")
print("rank      x      y      sigma      radius")
for rank, c in enumerate(q1['largest'], 1):
    print(f"{rank:4d} {c['x']:6d} {c['y']:6d} {c['sigma']:10.3f} {c['radius']:11.3f}")
```

Scale-normalized LoG: 60 dark-centre detections





```
Retained candidate centres: 60
Sigma search range: [1.5, 18.0] (reduced-image pixels)
Five largest detected central disks:
Coordinates, sigma and radii below are in pixels of the 360 x 360 reduced image.
rank      x      y      sigma      radius
   1    282    338      6.961       9.844
   2    106    255      6.014       8.505
   3    179    327      6.014       8.505
   4    178    261      5.590       7.906
   5    189    240      4.490       6.349
```

Interpretation

The largest foreground flower disks are recovered at larger scales than distant centres. Parameters refer to the 360 × 360 image; multiply coordinates and radii approximately by four for the original image. Irregular shape, occlusion and dark leaves surrounded by petals can still cause errors. There is no supplied annotation set, so the candidate count is not a validated count of every flower.

2. RANSAC line and circle fitting

The point set contains 50 noisy circle points and 50 noisy line points, generated with seed 0. The line is represented by **ax + by = d**, where $a^2 + b^2 = 1$ and $d \geq 0$. Line error is the perpendicular distance; circle error is the absolute radial distance.

RANSAC uses two points for a line and three for a circle. Each search tries 5,000 hypotheses with an error threshold of 1.5 units. The minimum consensus sizes are 30 for the line and 25 for the circle after removing the line consensus. The selected consensuses are kept fixed during refinement.

Minimal models and geometric errors

In [5]:
```python
def line_model(p):
    v=p[1]-p[0]
    if np.linalg.norm(v)<1e-10: return None
    n=np.array([-v[1],v[0]])/np.linalg.norm(v); d=n@p[0]
    if d<0: n,d=-n,-d
    return np.r_[n,d]
```

In [6]:
```python
def circle_model(p):
    a=2*(p[1:]-p[0]); b=(p[1:]**2).sum(1)-(p[0]**2).sum()
    if abs(np.linalg.det(a))<1e-8: return None
    c=np.linalg.solve(a,b); return np.r_[c,np.linalg.norm(p[0]-c)]
```

In [7]:
```python
def residual(x,m,kind):
    return abs(x@m[:2]-m[2]) if kind=='line' else abs(np.linalg.norm(x-m[:2],axis=1)-m[2])
```

RANSAC and consensus refinement

Hypotheses are ranked by consensus size; truncated squared residual breaks ties. TLS refines the line under the unit-normal constraint. Nonlinear least squares minimizes squared radial residuals for the circle. Degenerate samples are rejected.

In [8]:
```
def ransac(x,kind,seed,threshold=1.5,iterations=5000,min_consensus=30):
    rng=np.random.default_rng(seed); size=2 if kind=='line' else 3
    best=None; score=(-1,-np.inf)
    for _ in range(iterations):
        ids=rng.choice(len(x),size,replace=False)
        m=(line_model if kind=='line' else circle_model)(x[ids])
        if m is None: continue
        e=residual(x,m,kind); mask=e<threshold
        key=(int(mask.sum()),-float(np.sum(np.minimum(e,threshold)**2)))
        if key[0]>=min_consensus and key>score: best=(m,mask,ids); score=key
    if best is None: raise RuntimeError('No consensus')
    sample,mask,ids=best; pts=x[mask]
    if kind=='line':
        mean=pts.mean(0); _,_,vt=np.linalg.svd(pts-mean); n=vt[-1]; d=n@mean
        if d<0: n,d=-n,-d
        fit=np.r_[n,d]
    else:
        fit=least_squares(lambda m:np.linalg.norm(pts-m[:2],axis=1)-m[2],sample,bounds=([-
np.inf,-np.inf,1e-8],[np.inf]*3)).x
    return sample,fit,mask,ids
```

2(a–c). Fitted models and consensus points

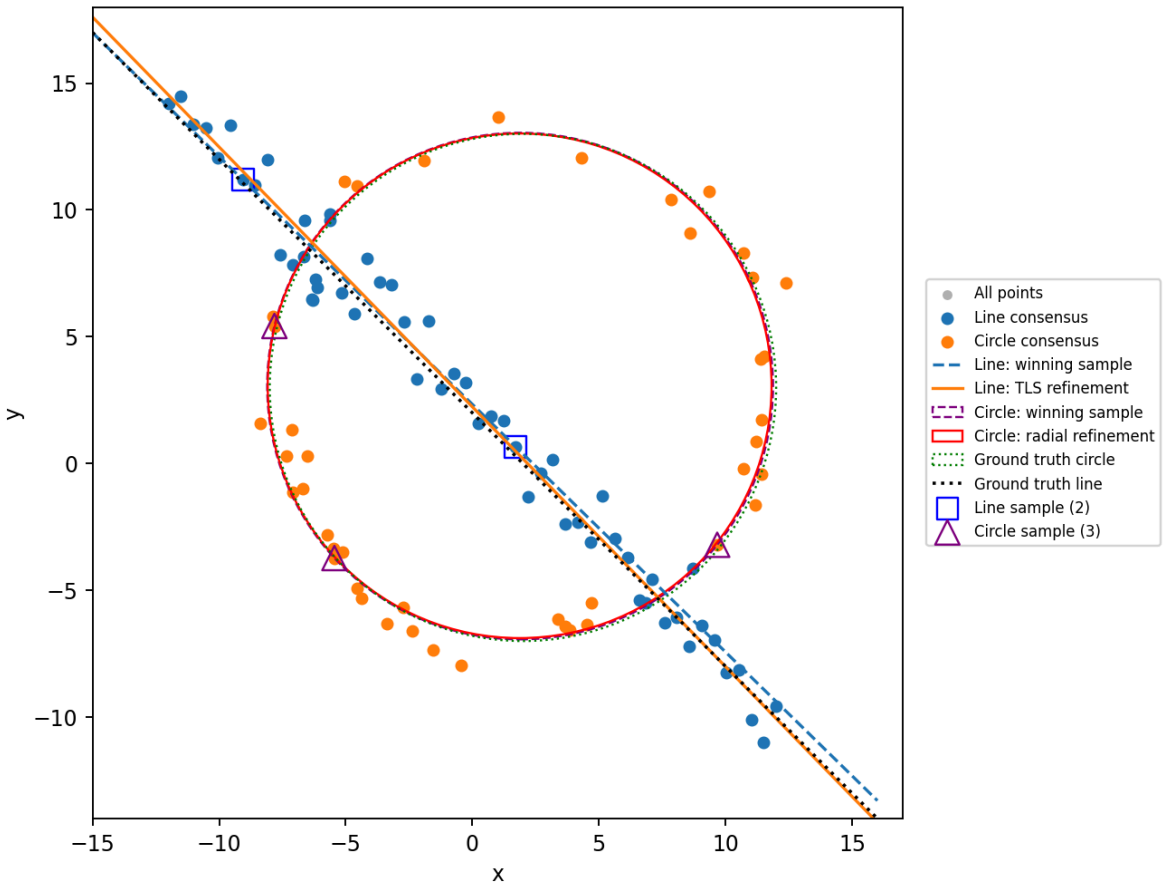
The plot shows the point set, the winning samples and their models, both consensuses, the refined line and circle, and the ground-truth models.

In [9]:
```
def fitting():
    # Match Listing 1's legacy RNG exactly, with the suggested seed enabled.
    np.random.seed(0); n=50; t=np.random.uniform(0,2*np.pi,n);
noise=.625*np.random.randn(n)
    circ=np.c_[2+(10+noise)*np.cos(t),3+(10+noise)*np.sin(t)]
    xx=np.linspace(-12,12,n); line=np.c_[xx,-xx+2+np.random.randn(n)]
    x=np.vstack([circ,line]); ls,lf,lmask,li=ransac(x,'line',10)
    remaining=x[~lmask]; cs,cf,cmask,ci=ransac(remaining,'circle',20,min_consensus=25)
    first=ransac(x,'circle',20,min_consensus=25)
    fig,ax=plt.subplots(figsize=(10,6)); ax.scatter(*x.T,s=13,c='.7',label='All points')
    ax.scatter(*x[lmask].T,s=24,c='tab:blue',label='Line consensus')
    ax.scatter(*remaining[cmask].T,s=24,c='tab:orange',label='Circle consensus')
    u=np.array([-15,16])
    for m,style,label in [(ls,'--','Line: winning sample'),(lf,'-','Line: TLS
refinement')]: ax.plot(u,(m[2]-m[0]*u)/m[1],style,label=label,lw=1.4)
    for m,style,color,label in [(cs,'--','purple','Circle: winning sample'),(cf,'-
','red','Circle: radial refinement'),(np.array([2,3,10]),':','green','Ground truth
circle')]: ax.add_patch(plt.Circle(m[:2],m[2],fill=False,ls=style,color=color,label=label))
    ax.plot(u,-u+2,':',color='black',label='Ground truth line')
    ax.scatter(*x[li].T,marker='s',s=100,facecolors='none',edgecolors='blue',label='Line
sample (2)')

ax.scatter(*remaining[ci].T,marker='^',s=130,facecolors='none',edgecolors='purple',label='C
ircle sample (3)')
    ax.set(xlim=(-15,17),ylim=(-14,18),aspect='equal',xlabel='x',ylabel='y');
ax.legend(fontsize=7,loc='center left',bbox_to_anchor=(1.02,.5)); savefig('q2_ransac.png')
    np.savez(OUT/'q2_data.npz',points=x,line_mask=lmask,circle_remaining_mask=cmask)
    return
{'seed':0,'threshold':1.5,'iterations':5000,'line_min_consensus':30,'circle_min_consensus':
25,'line_sample_model':ls.tolist(),'line_refined':lf.tolist(),'line_consensus':int(lmask.su
m()),'line_circle_contamination':int(lmask[:50].sum()),'circle_sample_model':cs.tolist(),'c
ircle_refined':cf.tolist(),'circle_consensus':int(cmask.sum()),'circle_first_model':first[1
].tolist(),'circle_first_consensus':int(first[2].sum()),'circle_first_line_contamination':i
nt(first[2][50:].sum())}
```

In [10]:
```
q2 = fitting()
print('Line refined [a, b, d]:', np.round(q2['line_refined'], 6))
print('Normal length:', np.linalg.norm(q2['line_refined'][:2]))
print('Line consensus:', q2['line_consensus'])
```

```
print('Circle refined [cx, cy, r]:', np.round(q2['circle_refined'], 6))
print('Circle consensus after line removal:', q2['circle_consensus'])
print('Ground truth: line y = -x + 2; circle centre (2, 3), radius 10.')
```



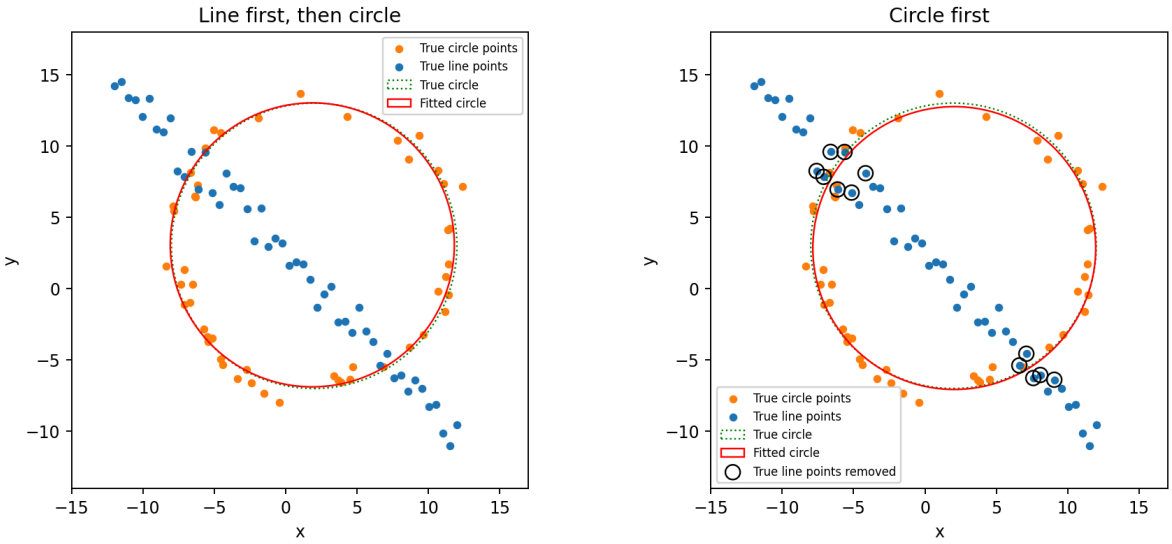

```
Line refined [a, b, d]: [0.715649 0.69846  1.561775]
Normal length: 0.9999999999999999
Line consensus: 57
Circle refined [cx, cy, r]: [1.86959  3.065497 9.949293]
Circle consensus after line removal: 43
Ground truth: line y = -x + 2; circle centre (2, 3), radius 10.
```

2(d). Fitting the circle first

A circle fit can include line points near the intersections. A very large circle can also approximate a line over a short arc. The following comparison shows the circle-first fit and the true line points it removes. Ground-truth labels are used only to assess the result.

In [11]:
```
points = np.load(OUT/'q2_data.npz')['points']
_, first_circle, first_mask, _ = ransac(points, 'circle', seed=20, min_consensus=25)
fig, axes = plt.subplots(1, 2, figsize=(11, 4.8))
for ax, title, fit in [
    (axes[0], 'Line first, then circle', q2['circle_refined']),
    (axes[1], 'Circle first', first_circle),
]:
    ax.scatter(*points[:50].T, s=15, c='tab:orange', label='True circle points')
    ax.scatter(*points[50:].T, s=15, c='tab:blue', label='True line points')
    ax.add_patch(plt.Circle((2, 3), 10, fill=False, color='green', ls=':', label='True
circle'))
    ax.add_patch(plt.Circle(fit[:2], fit[2], fill=False, color='red', label='Fitted
circle'))
    ax.set(xlim=(-15, 17), ylim=(-14, 18), aspect='equal', title=title, xlabel='x',
ylabel='y')
removed_line = first_mask & (np.arange(len(points)) >= 50)
axes[1].scatter(*points[removed_line].T, facecolors='none', edgecolors='black',
                s=80, label='True line points removed')
for ax in axes:
```

```
    ax.legend(fontsize=7)
savefig('q2_order_comparison.png')
print('Circle-first [cx, cy, r]:', np.round(first_circle, 6))
print('Circle-first consensus:', int(first_mask.sum()))
print('True line points removed:', int(removed_line.sum()))
```




```
Circle-first [cx, cy, r]: [2.055644 2.843251 9.908802]
Circle-first consensus: 61
True line points removed: 12
```

Interpretation and threshold choice

For slope −1 and vertical noise σ = 1, orthogonal line noise has standard deviation 1/√2. The threshold 1.5 is about 2.12 such standard deviations, and about 2.4 circle radial-noise standard deviations (σ = 0.625). Removing a line consensus also removes some genuine circle points near intersections. In this seed, fitting the circle first still finds a plausible circle but removes 12 genuine line points.

3. Planar homography and blending

The image pairs use NASA/JPL travel posters in two outdoor advertising displays. Four destination corners are selected in top-left, top-right, bottom-right, bottom-left order. A homography maps the poster corners to these points, and a warped mask limits the alpha blend to the display. The Grand Tour poster is centred on a landscape canvas to preserve its proportions.

The space-tourism theme suits the advertising displays. The Mars poster also contrasts with the pale street scene. Image sources and credits. The supplied Wadham and flag pair is included for comparison.

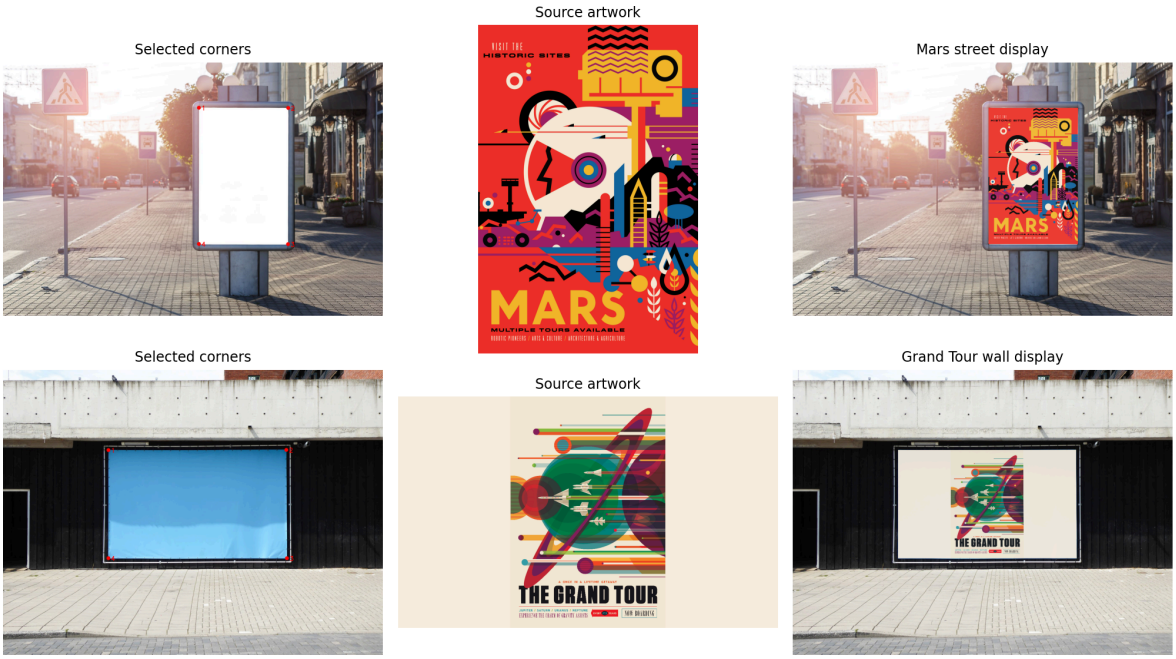
In [12]:
```
def composite():
    base=cv.imread(str(DATA/'wadham.jpg')); flag=cv.imread(str(DATA/'flag.png'))
    dst=np.float32([[160,225],[338,262],[337,335],[159,304]])
    h,w=flag.shape[:2]; src=np.float32([[0,0],[w-1,0],[w-1,h-1],[0,h-1]])
    H=cv.getPerspectiveTransform(src,dst)
    example_H=H.copy()
    warped=cv.warpPerspective(flag,H,(base.shape[1],base.shape[0]))
    mask=cv.warpPerspective(np.ones((h,w),np.float32),H,(base.shape[1],base.shape[0]))
[:,:,None]
    result=np.uint8(base*(1-.8*mask)+warped*.8*mask)
```

```
    cv.imwrite(str(OUT/'q3_wadham.png'),result)
    records=[]
    pairs=[('pair1_background.jpg','pair1_overlay_mars.jpg',[[928,216],[1351,217],
[1350,861],[928,860]],'Mars street display'),
           ('pair2_background.jpg','pair2_overlay_grand_tour.jpg',[[499,381],[1342,380],
[1345,892],[499,892]],'Grand Tour wall display')]
    fig,axes=plt.subplots(2,3,figsize=(12,7))
    for row,(bg,fg,corners,title) in enumerate(pairs):
        base=cv.imread(str(ROOT/'my_images'/bg)); art=cv.imread(str(ROOT/'my_images'/fg))
        corner_file=ROOT/'my_images'/'corners.json'
        saved=json.loads(corner_file.read_text()) if corner_file.exists() else {}
        dst=np.float32(saved.get(bg,corners))
        # Landscape panel: preserve the entire portrait poster on a cream canvas.
        if row==1:
            canvas=np.full((900,1480,3),(220,235,246),np.uint8)
            scaled=cv.resize(art,(int(art.shape[1]*900/art.shape[0]),900))
            left=(1480-scaled.shape[1])//2; canvas[:,left:left+scaled.shape[1]]=scaled;
art=canvas
        h,w=art.shape[:2]; src=np.float32([[0,0],[w-1,0],[w-1,h-1],[0,h-1]])
        H=cv.getPerspectiveTransform(src,dst); size=(base.shape[1],base.shape[0])
        warped=cv.warpPerspective(art,H,size)
        mask=cv.warpPerspective(np.ones((h,w),np.float32),H,size)[:,:,None]
        # Slight transparency retains a little local illumination from the photo.
        alpha=.94; result=np.uint8(np.clip(base*(1-alpha*mask)+warped*alpha*mask,0,255))
        cv.imwrite(str(OUT/f'q3_pair{row+1}.jpg'),result)
        marked=base.copy()
        for i,p in enumerate(dst.astype(int)):
            cv.circle(marked,tuple(p),9,(0,0,255),-1);
cv.putText(marked,str(i+1),tuple(p+12),cv.FONT_HERSHEY_SIMPLEX,1,(0,0,255),2)
        for ax,img,label in zip(axes[row],[marked,art,result],['Selected corners','Source
artwork',title]):
            ax.imshow(cv.cvtColor(img,cv.COLOR_BGR2RGB)); ax.set_title(label,fontsize=10);
ax.axis('off')

records.append({'background':bg,'overlay':fg,'destination_points':dst.tolist(),'homography'
:H.tolist(),'alpha':alpha})
    savefig('q3_pairs.png')
    return
{'example_homography':example_H.tolist(),'pairs':records,'corner_selection':'Manually
selected from displayed photos; saved coordinates in TL, TR, BR, BL order.'}
```

In [13]:

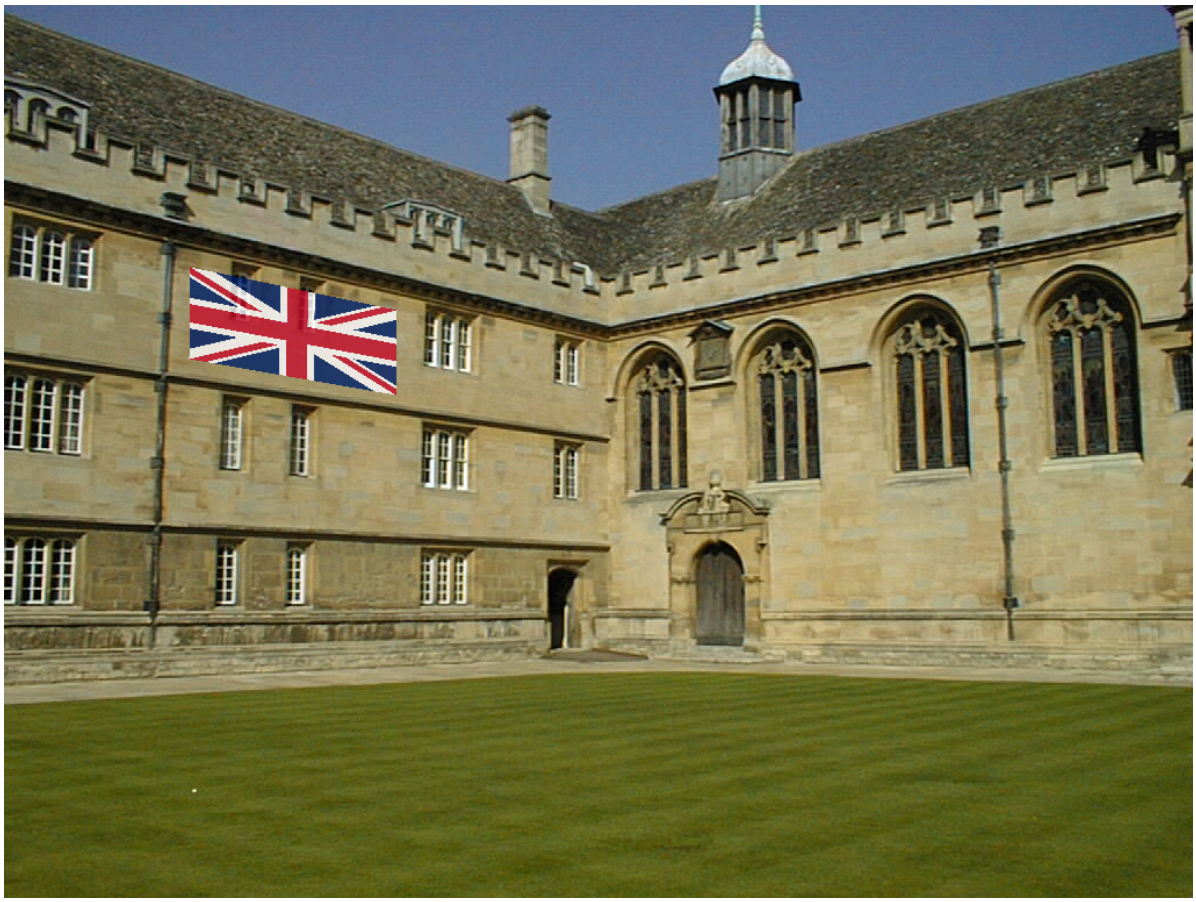
```
q3 = composite()
for i, pair in enumerate(q3['pairs'], 1):
    print(f"Pair {i}: {pair['overlay']} -> {pair['background']}")
    print('Destination corners:', np.asarray(pair['destination_points']))
    print('Homography:')
    print(np.array2string(np.array(pair['homography']), precision=7))
    print()
display(Markdown('**Supplied Wadham/flag example:**'))
display(Image(filename=str(OUT/'q3_wadham.png'), width=800))
```




```
Pair 1: pair1_overlay_mars.jpg -> pair1_background.jpg
Destination corners: [[ 928.  216.]
 [1351.  217.]
 [1350.  861.]
 [ 928.  860.]]
Homography:
[[ 1.6231582e-01  5.6588884e-04  9.2800000e+02]
 [ 3.8342346e-04  1.6624753e-01  2.1600000e+02]
 [-1.4119707e-09  6.0979401e-07  1.0000000e+00]]

Pair 2: pair2_overlay_grand_tour.jpg -> pair2_background.jpg
Destination corners: [[ 499.  381.]
 [1342.  380.]
 [1345.  892.]
 [ 499.  892.]]
Homography:
[[ 5.6820751e-01 -1.9644577e-03  4.9900000e+02]
 [-1.1779496e-03  5.6489773e-01  3.8100000e+02]
 [-1.3205713e-06 -3.9367890e-06  1.0000000e+00]]
```

Supplied Wadham/flag example:



Interpretation

Both displays are nearly frontal, so the projective distortion is modest. A single homography assumes one plane and cannot reproduce fabric wrinkles, curved surfaces or foreground occluders. Blending at alpha 0.94 retains a little background illumination, but does not physically reproduce shadows or reflections.

4. PCB alignment and image differences

AECC1.jpg is the template. All five other PCB images are processed using SIFT (up to 12,000 features), BFMatcher/L2 and the nearest-neighbour ratio test (0.75). Homography RANSAC uses a 3-pixel threshold, up to 20,000 trials and confidence 0.999.

Raw per-channel absolute differences are saved losslessly as PNG. A warped support mask, eroded by three pixels, excludes unsupported borders. The overview shows matches, aligned images and raw differences for every comparison.

In [14]:
```
def pcb():
    cv.setRNGSeed(0); template=cv.imread(str(DATA/'pcbs'/'AECC1.jpg'));
gray=cv.cvtColor(template,cv.COLOR_BGR2GRAY)
    sift=cv.SIFT_create(nfeatures=12000); kt,dt=sift.detectAndCompute(gray,None); records=
[]
    fig,axes=plt.subplots(5,3,figsize=(10,14))
    for row,path in enumerate(p for p in sorted((DATA/'pcbs').glob('*.jpg')) if
p.name!='AECC1.jpg'):
        im=cv.imread(str(path));
ki,di=sift.detectAndCompute(cv.cvtColor(im,cv.COLOR_BGR2GRAY),None)
        pairs=cv.BFMatcher(cv.NORM_L2).knnMatch(di,dt,k=2); good=[a for a,b in pairs if
a.distance<.75*b.distance]
```

```
        if len(good)<4: raise RuntimeError(f'Insufficient matches: {path.name}')
        src=np.float32([ki[m.queryIdx].pt for m in good]);
dst=np.float32([kt[m.trainIdx].pt for m in good])
        H,inliers=cv.findHomography(src,dst,cv.RANSAC,3.0,maxIters=20000,confidence=.999)
        if H is None or inliers is None: raise RuntimeError(f'No homography: {path.name}')
        valid=inliers.ravel().astype(bool)
        if H is None or valid.sum()<10: raise RuntimeError(f'Unreliable homography:
{path.name}')
        size=(template.shape[1],template.shape[0]); aligned=cv.warpPerspective(im,H,size)

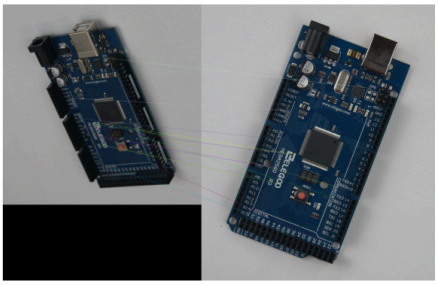
support=cv.warpPerspective(np.ones(im.shape[:2],np.uint8),H,size,flags=cv.INTER_NEAREST)
        support=cv.erode(support,np.ones((7,7),np.uint8)).astype(bool)
        diff=cv.absdiff(aligned,template); diff[~support]=0
        matches=cv.drawMatches(im,ki,template,kt,[good[i] for i in np.flatnonzero(valid)
[:60]],None,flags=cv.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
        stem=path.stem
        for name,img in [('aligned',aligned),('matches',matches)]:
cv.imwrite(str(OUT/f'q4_{stem}_{name}.jpg'),img)
        cv.imwrite(str(OUT/f'q4_{stem}_difference.png'),diff)
        cv.imwrite(str(OUT/f'q4_{stem}_support.png'),support.astype(np.uint8)*255)
        for ax,img,title in zip(axes[row],[matches,aligned,diff],[f'{stem}: inlier
matches','Aligned','Absolute difference']): ax.imshow(cv.cvtColor(img,cv.COLOR_BGR2RGB));
ax.set_title(title,fontsize=9); ax.axis('off')
        projected=cv.perspectiveTransform(src.reshape(-1,1,2),H).reshape(-1,2)

records.append({'image':path.name,'ratio_matches':len(good),'ransac_inliers':int(valid.sum(
)),'median_reprojection_px':float(np.median(np.linalg.norm(projected[valid]-
dst[valid],axis=1))),'overlap_fraction':float(support.mean()),'overlap_mae':float(diff[supp
ort].mean()),'homography':H.tolist()})
    savefig('q4_overview.png')
    cv.imwrite(str(OUT/'q4_AECC1_difference.png'),np.zeros_like(template))
    return records
```

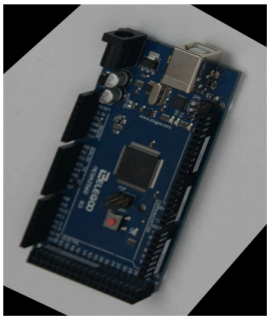
In [15]:

```
q4 = pcb()
print("Image        Matches   H-inliers   Median reproj(px)    Overlap MAE")
for row in q4:
    print(f"{row['image']:12s} {row['ratio_matches']:7d} {row['ransac_inliers']:11d}"
          f" {row['median_reprojection_px']:19.3f} {row['overlap_mae']:14.3f}")
```

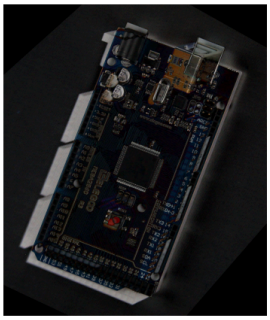
AAAA1: inlier matches



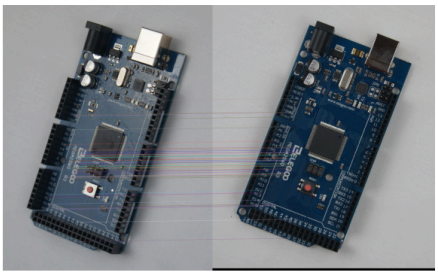
Aligned



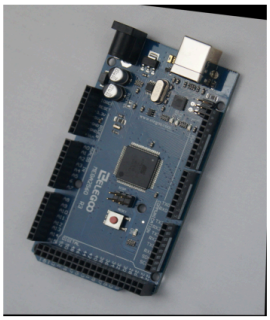
Absolute difference



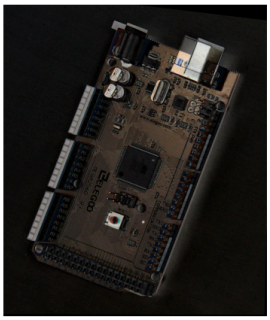
ADAD1: inlier matches



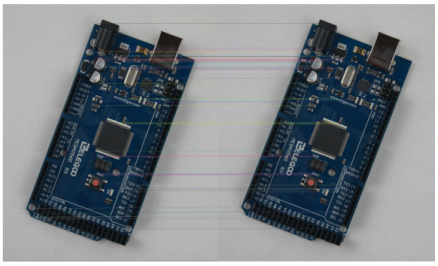
Aligned



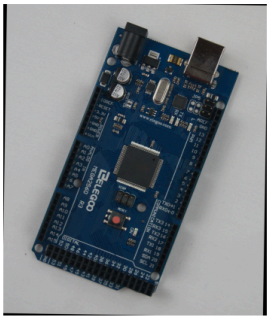
Absolute difference



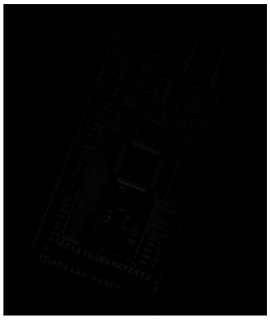
AECC2: inlier matches



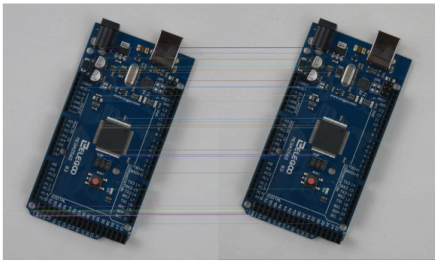
Aligned



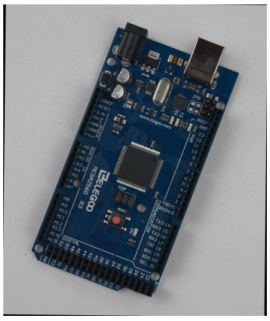
Absolute difference



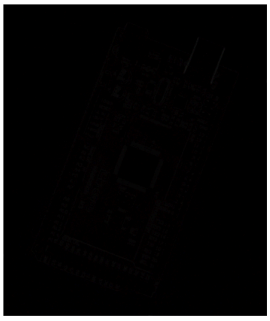
AECC3: inlier matches



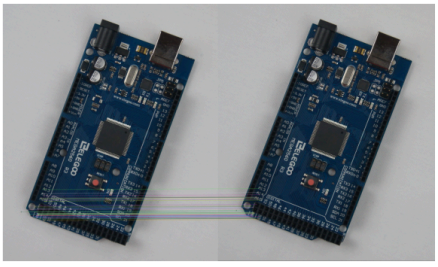
Aligned



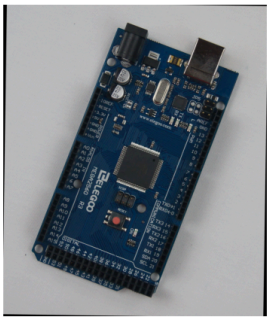
Absolute difference



AECC4: inlier matches



Aligned



Absolute difference

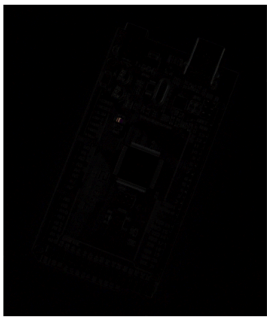
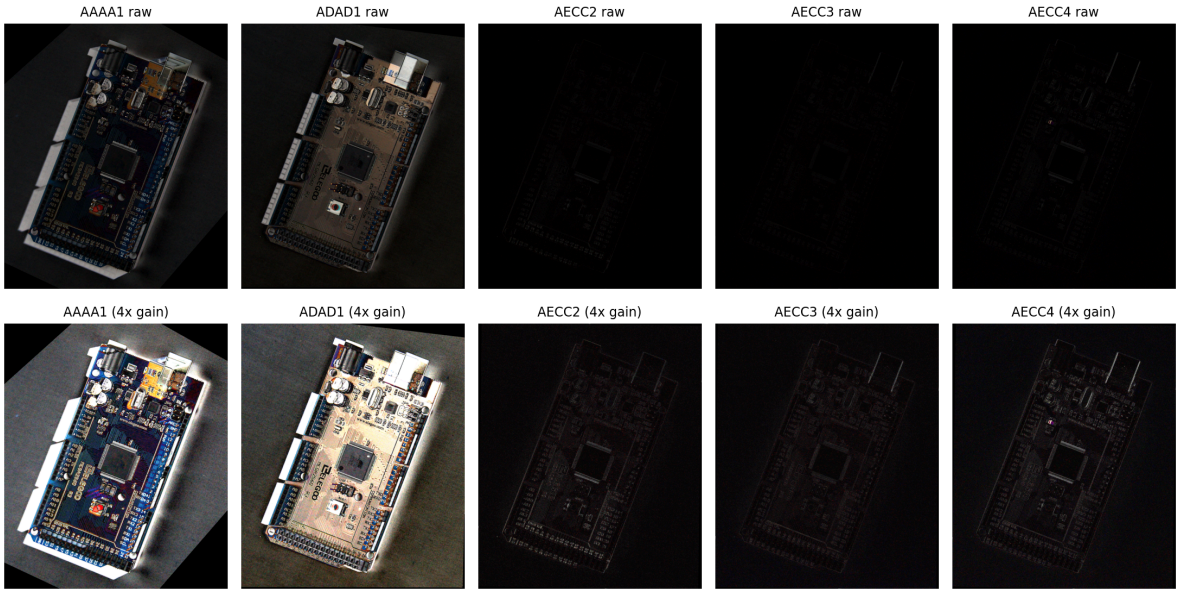


Image	Matches	H-inliers	Median reproj(px)	Overlap MAE
AAAA1.jpg	57	14	1.376	28.987
ADAD1.jpg	339	190	0.775	30.632
AECC2.jpg	4095	3583	0.366	3.520
AECC3.jpg	5071	4509	0.304	3.299
AECC4.jpg	3608	3167	0.403	4.070

Raw differences and enhanced views for all PCB images

The first row is the actual absolute difference. The second uses **4× display gain**, clipped to [0,255], to make small residuals visible. Gain is not used to compute MAE. The template self-difference is checked separately.

In [16]:
```
fig, axes = plt.subplots(2, len(q4), figsize=(13, 6.8))
for col, row in enumerate(q4):
    stem = Path(row['image']).stem
    raw = cv.imread(str(OUT/f'q4_{stem}_difference.png'))
    for plot_row, gain in enumerate([1, 4]):
        shown = np.clip(raw.astype(float)*gain, 0, 255).astype(np.uint8)
        axes[plot_row, col].imshow(cv.cvtColor(shown, cv.COLOR_BGR2RGB))
        axes[plot_row, col].set_title(stem + (' raw' if gain == 1 else ' (4x gain)'),
fontsize=10)
        axes[plot_row, col].axis('off')
savefig('q4_differences.png')
self_difference = cv.imread(str(OUT/'q4_AECC1_difference.png'))
assert not self_difference.any()
print('AECC1 template self-difference: exactly zero.')
```




```
AECC1 template self-difference: exactly zero.
```

4(e). Why do the USB port, capacitors and reset button differ strongly?

These components protrude above the board. Viewpoint changes create parallax that one homography fitted to the board plane cannot remove. Metallic reflections, shadows, illumination changes and occlusion also affect the difference images.

AAAA1 and ADAD1 have overlap MAE values of about 29.0 and 30.6, compared with 3.3-4.1 for AECC2-AECC4. Their broad, bright differences probably reflect different board variants and/or different lighting conditions, rather than isolated component changes. This is an interpretation of the images, not a confirmed identification of board variants. Their maps should not be treated as evidence of individual faulty components.

AAAA1 also has only 14 homography inliers, so registration error may contribute. Low reprojection error at the matched features does not guarantee that every raised component aligns.

Numerical checks

Check the line-normal constraint, circle parameters, corner mappings and valid regions of the difference images.

In [17]:
```
assert np.isclose(np.linalg.norm(q2['line_refined'][:2]), 1)
assert np.linalg.norm(np.array(q2['circle_refined'][:2]) - [2, 3]) < 1
assert abs(q2['circle_refined'][2] - 10) < 1
assert len(q4) == 5
assert all(row['median_reprojection_px'] < 3 for row in q4)
for i, pair in enumerate(q3['pairs']):
    art = cv.imread(str(ROOT/'my_images'/pair['overlay']))
    h, w = art.shape[:2]
    if i == 1:
        h, w = 900, 1480
    corners = np.float32([[0, 0], [w-1, 0], [w-1, h-1], [0, h-1]])
    projected = cv.perspectiveTransform(corners.reshape(-1, 1, 2),
                                        np.array(pair['homography'])).reshape(-1, 2)
    assert np.max(np.linalg.norm(projected-pair['destination_points'], axis=1)) < 1e-3
for row in q4:
    stem = Path(row['image']).stem
    diff = cv.imread(str(OUT/f'q4_{stem}_difference.png'))
    support = cv.imread(str(OUT/f'q4_{stem}_support.png'), cv.IMREAD_GRAYSCALE) > 0
    assert not diff[~support].any()
metrics = {'q1': q1, 'q2': q2, 'q3': q3, 'q4': q4}
(OUT/'metrics.json').write_text(json.dumps(metrics, indent=2))
print('All notebook numerical checks passed.')
print('Measurements saved to results/metrics.json')
```

```
All notebook numerical checks passed.
Measurements saved to results/metrics.json
```